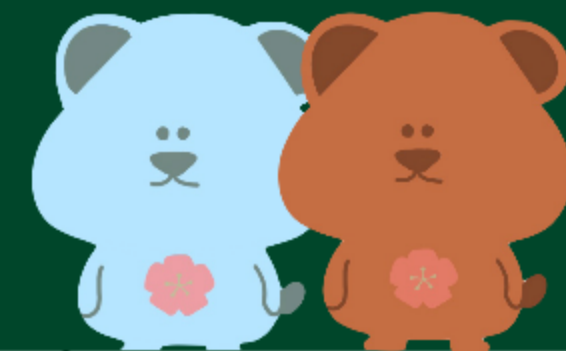




DL 5장

3팀 | 김기연 서영 이윤지

5.1 합성곱 신경망



합성곱 신경망 개념과 필요성

#1 합성곱 신경망 개념

4장에서 배운 딥러닝의 역전파는 순전파 과정에서 생긴 오차 정보가 출력층, 은닉층, 입력층을 거치며 전송됨.

복잡한 계산과 많은 자원(GPU, 메모리 등)이 필요함.

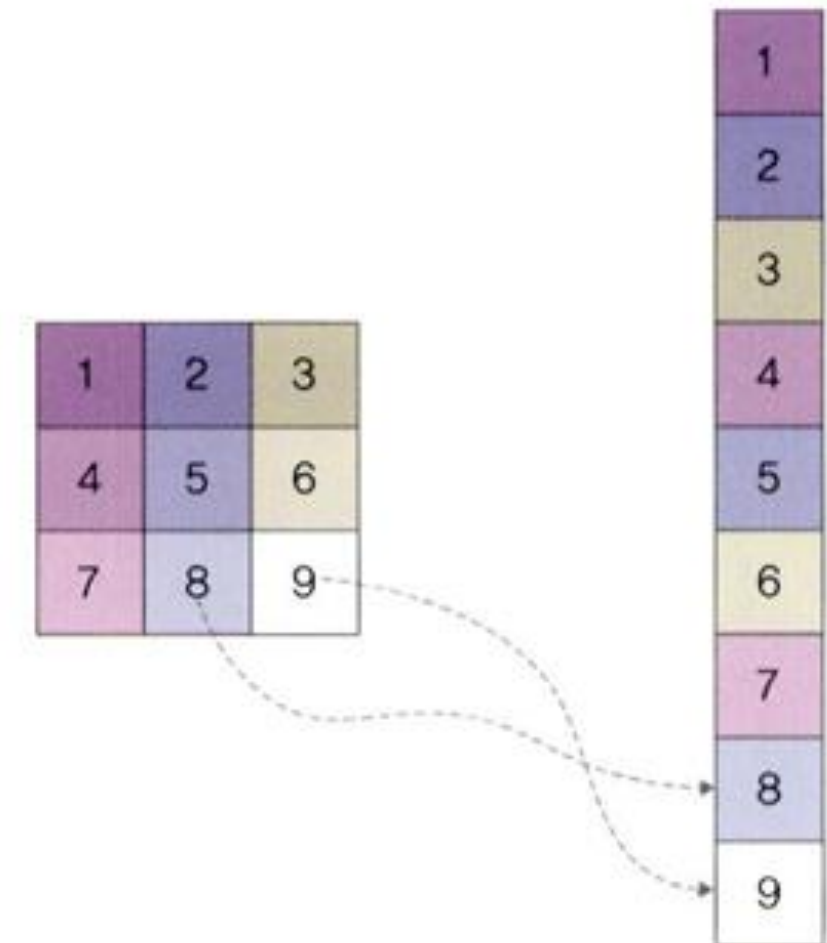
→ 합성곱 신경망(CNN)은 이미지 전체를 한 번에 계산하지 않고, 국소적 부분을 계산

→ 시간과 자원을 절약, 이미지의 세밀한 부분까지 분석

#2 합성곱 신경망의 필요성

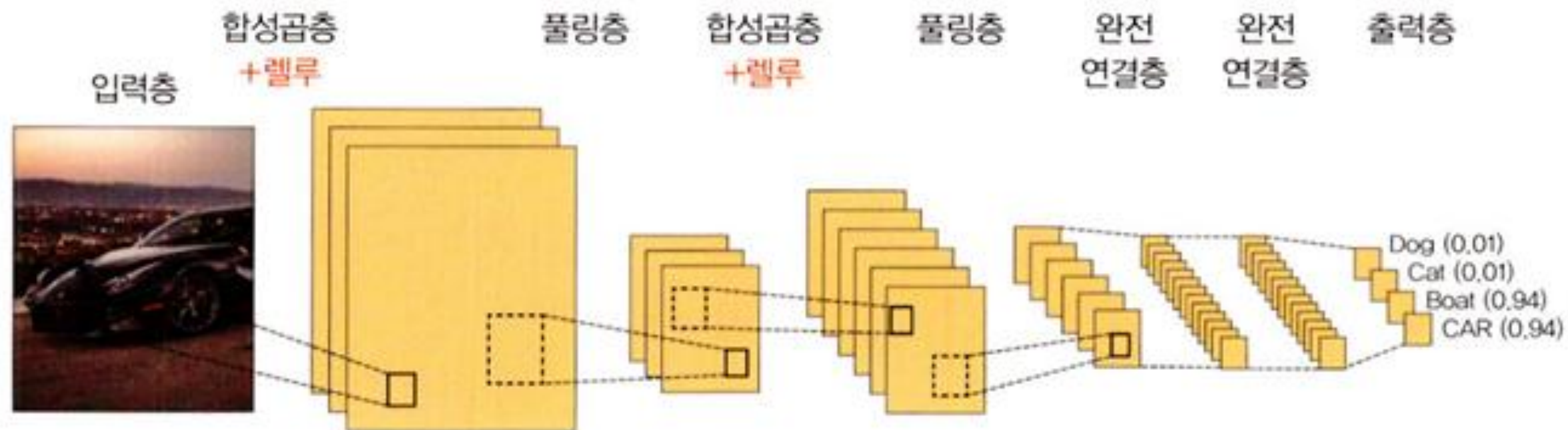
이미지 분석의 경우, 펼쳐서(flatten) 각 픽셀에 가중치를 곱하여 은닉층으로 전달하게 되면, 데이터의 공간적 구조가 무시됨.

→ 합성곱 신경망은 이를 해결!



합성곱 신경망 구조

#1 전체 구조: 입력층 → 합성곱층 → 풀링층 → 완전연결층 → 출력층

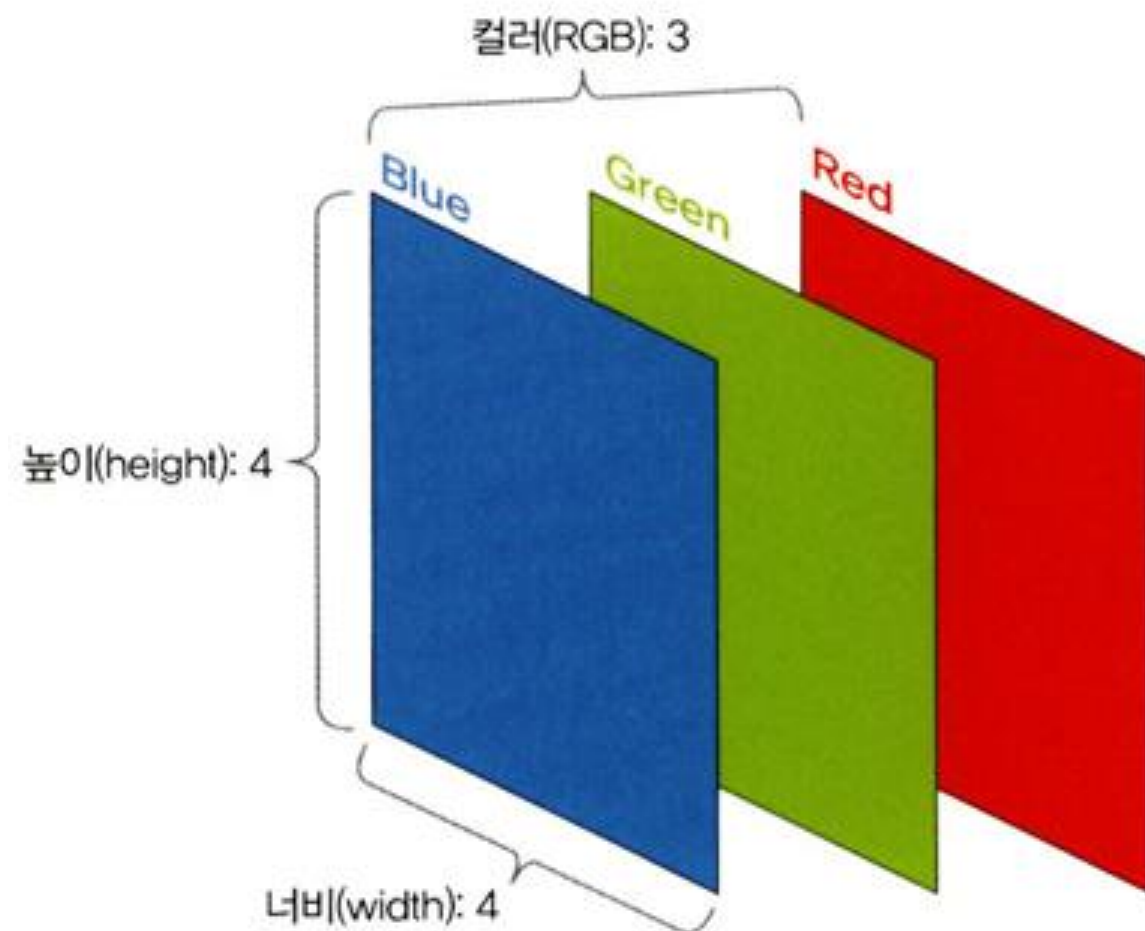


합성곱층과 풀링층을 거치면서 주요 특성 벡터(feature vector)를 추출
주요 특성 벡터는 완전 연결층을 통해 1차원 벡터로 변환
출력층에서 softmax 함수를 사용하여 최종 결과가 출력

합성곱 신경망 구조

입력층

- 입력 이미지 데이터가 들어오는 계층
- 이미지는 높이(height), 너비(width), 채널(channel)로 이루어진 3차원 데이터
- 채널값은 Grey scale이면 1, 컬러(RGB) 면 3



합성곱 신경망 구조

합성곱층 Convolutional layer

- 입력 데이터에서 특성 추출 → 커널/필터 → feature map 특성 맵
- 커널은 보통 3x3, 5x5 크기로 적용 + stride 에 따라 이동
- 예시) grey scale + 커널 3x3 + Stride가 1



$$1 \times 1 + 0 \times 0 + 0 \times 1 + 0 \times 0 + 1 \times 1 + 0 \times 0 + 0 \times 1 + 0 \times 0 + 1 \times 1 = 3$$

합성곱 신경망 구조

합성곱층 Convolutional layer

- 입력 데이터에서 특성 추출 → 커널/필터 → feature map 특성 맵
- 커널은 보통 3x3, 5x5 크기로 적용 + stride 에 따라 이동
- 예시) grey scale + 커널 3x3 + Stride가 1



$$0 \times 1 + 0 \times 0 + 0 \times 1 + 1 \times 0 + 0 \times 1 + 0 \times 0 + 0 \times 1 + 1 \times 0 + 1 \times 1 = 1$$

합성곱 신경망 구조

합성곱층 Convolutional layer

- 입력 데이터에서 특성 추출 → 커널/필터 → feature map 특성 맵
- 커널은 보통 3x3, 5x5 크기로 적용 + stride 에 따라 이동
- 예시) grey scale + 커널 3x3 + Stride가 1



$$0 \times 1 + 0 \times 0 + 0 \times 1 + 0 \times 0 + 0 \times 1 + 1 \times 0 + 1 \times 1 + 1 \times 0 + 0 \times 1 = 1$$

합성곱 신경망 구조

합성곱층 Convolutional layer

- 입력 데이터에서 특성 추출 → 커널/필터 → feature map 특성 맵
- 커널은 보통 3x3, 5x5 크기로 적용 + stride 에 따라 이동
- 예시) grey scale + 커널 3x3 + Stride가 1



$$0 \times 1 + 0 \times 0 + 1 \times 1 + 0 \times 0 + 1 \times 1 + 0 \times 0 + 1 \times 1 + 0 \times 0 + 0 \times 1 = 3$$

합성곱 신경망 구조

합성곱층 Convolutional layer

- 입력 데이터에서 특성 추출 → 커널/필터 → feature map 특성 맵
- 커널은 보통 3x3, 5x5 크기로 적용 + stride 에 따라 이동
- 예시) grey scale + 커널 3x3 + Stride가 1



$$0 \times 1 + 1 \times 0 + 0 \times 1 + 0 \times 0 + 0 \times 1 + 1 \times 0 + 1 \times 1 + 0 \times 0 + 0 \times 1 = 1$$

합성곱 신경망 구조

합성곱층 Convolutional layer

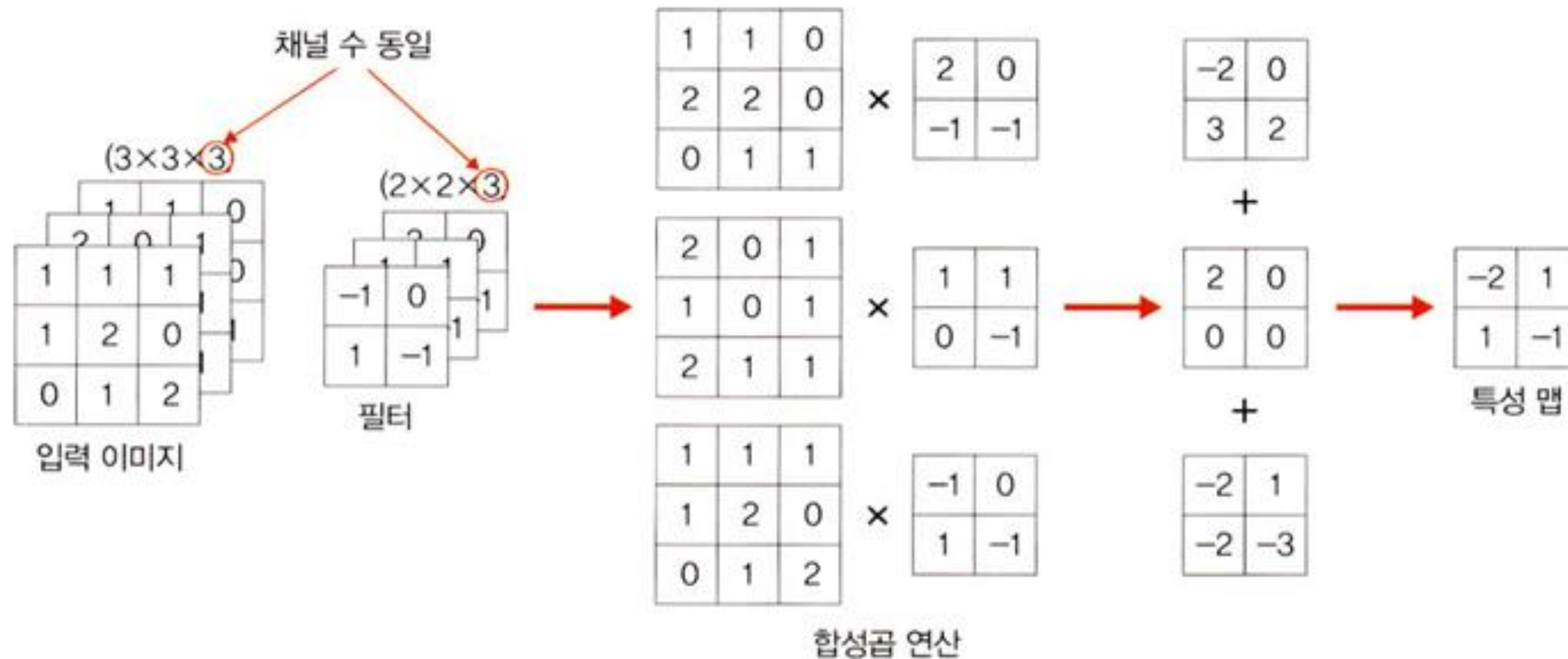
- 입력 데이터에서 특성 추출 → 커널/필터 → feature map 특성 맵
- 커널은 보통 3x3, 5x5 크기로 적용 + stride 에 따라 이동
- 예시) grey scale + 커널 3x3 + Stride가 1



합성곱 신경망 구조

합성곱층 Convolutional layer

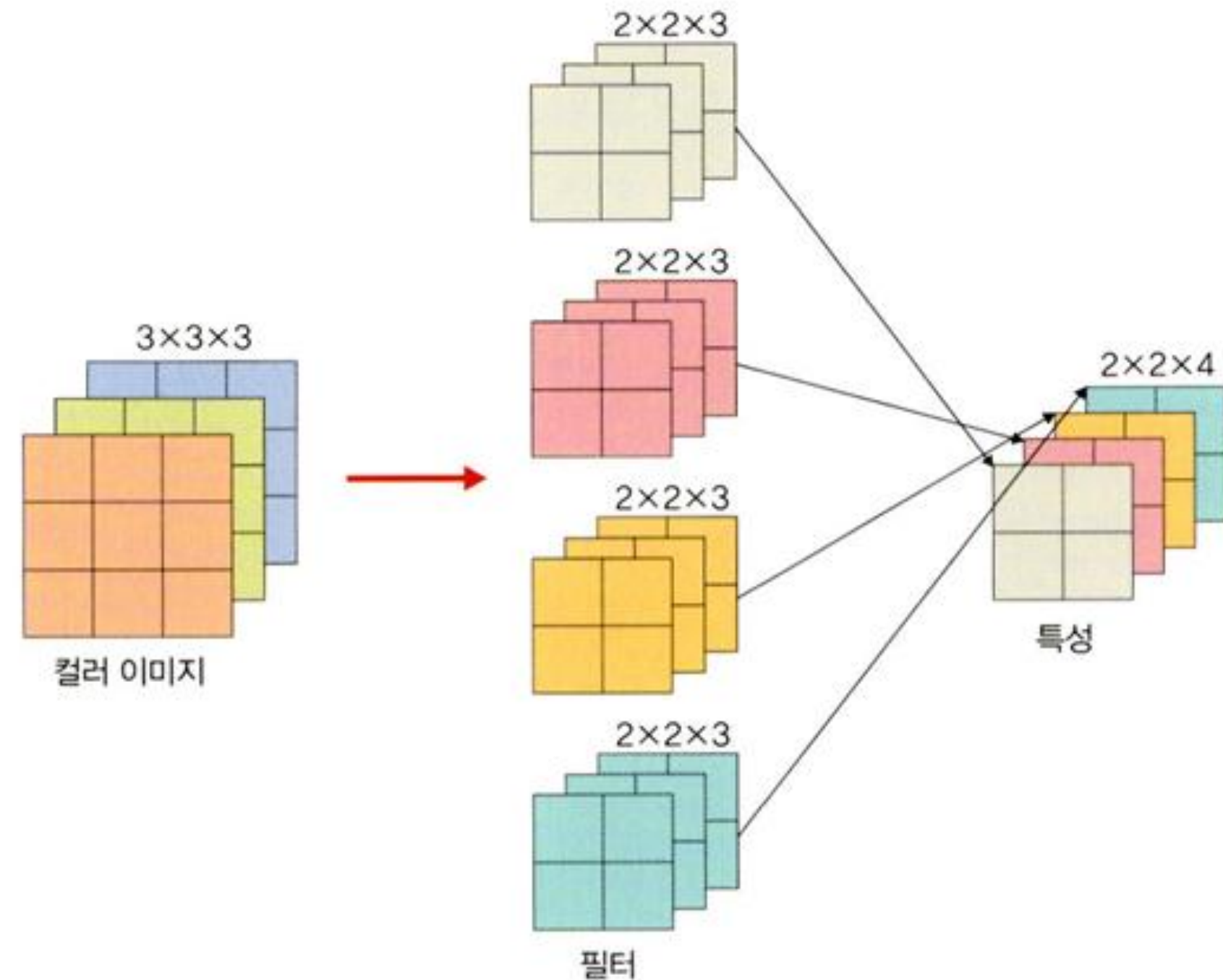
- 스트라이드 간격만큼 순회
- $(6,6,1) \rightarrow (4,4,1)$ 크기의 특성맵으로 줄어듦
- 예시) 컬러 이미지: 채널이 3개, RGB 각각에 서로 다른 가중치로 합성곱 $\rightarrow$ 결과를 더함



합성곱 신경망 구조

합성곱층 Convolutional layer

- 필터가 2개 이상이라면?
→ 필터 각각이 특성 추출 결과의 채널
- 입력데이터: $W1 \times H1 \times D1$
- 하이퍼파라미터:
 - K: 필터개수
 - F: 필터 크기
 - S: 스트라이드
 - P: 패딩
- 출력데이터
 - $W2 = (W1 - F + 2P) / S + 1$
 - $H2 = (H1 - F + 2P) / S + 1$
 - $D2 = K$



합성곱 신경망 구조

풀링층 Pooling layer

- 특성 맵의 차원을 다운샘플링 → 연산량 감소
 - 다운 샘플링: 이미지를 축소시키는 방법



- 1) 최대풀링: max pooling : 대상 영역에서 최댓값을 추출
- 2) 평균 풀링: average pooling : 대상 영역에서 평균을 반환

합성곱 신경망 구조

최대풀링 max pooling

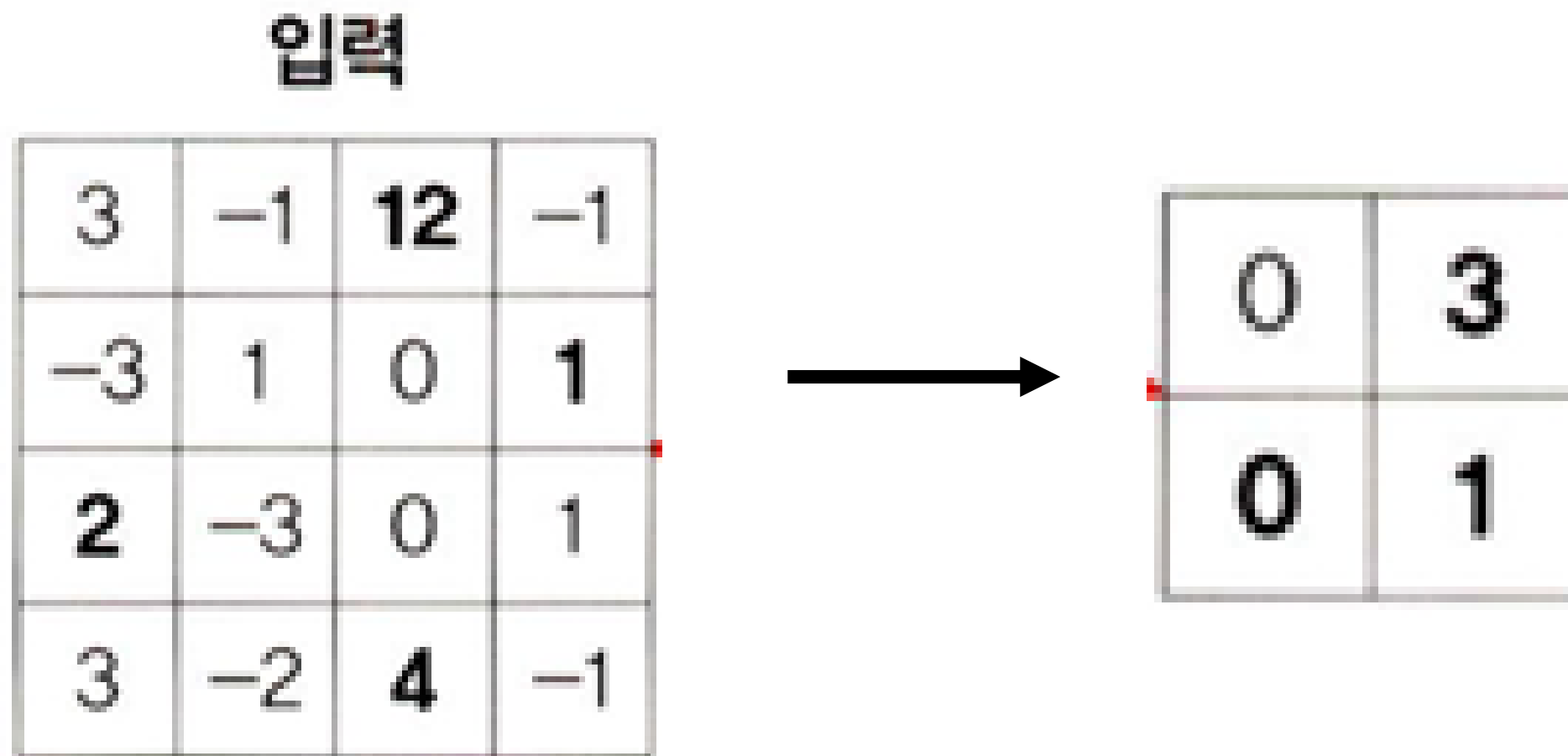
- 보통 최대 풀링을 사용



합성곱 신경망 구조

평균풀링 average pooling

- 단점: 각 커널 값을 평균화하여 중요한 가중치를 갖는 값의 특성이 희미해짐



합성곱 신경망 구조

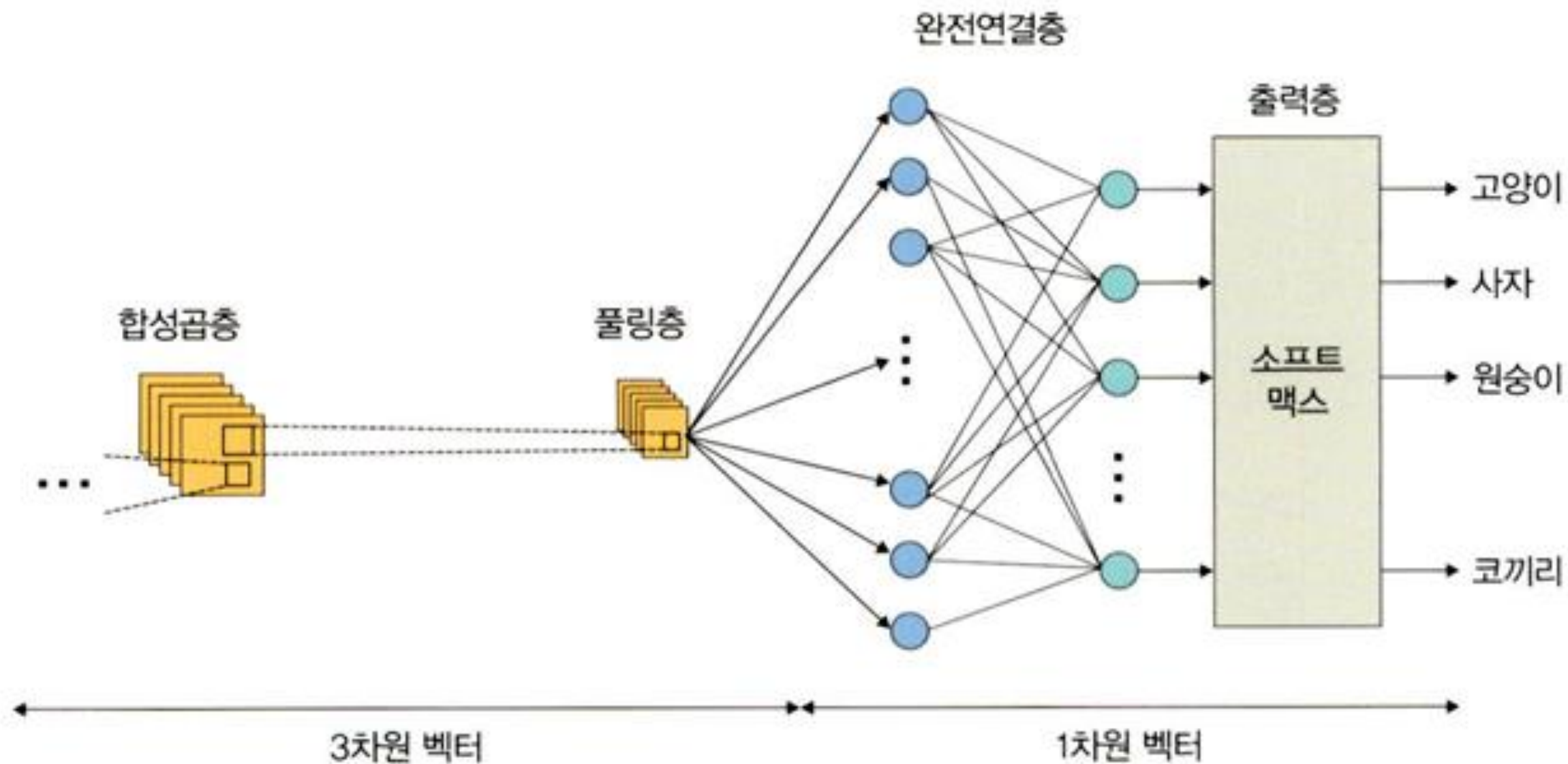
풀링 파라미터

- 입력 파라미터: $W1 \times H1 \times D1$
- 하이퍼파라미터
 - F: 필터크기
 - S: 스트라이드
- 출력 데이터
 - $W2 = (W1 - F) / S + 1$
 - $H2 = (H1 - F) / S + 1$
 - $D2 = D1$

합성곱 신경망 구조

완전 연결층 Fully connected layer

- 합성곱층과 풀링층을 거치면서 만들어진 3차원 특성맵을 1차원 벡터로 펼친다. flatten



합성곱 신경망 구조

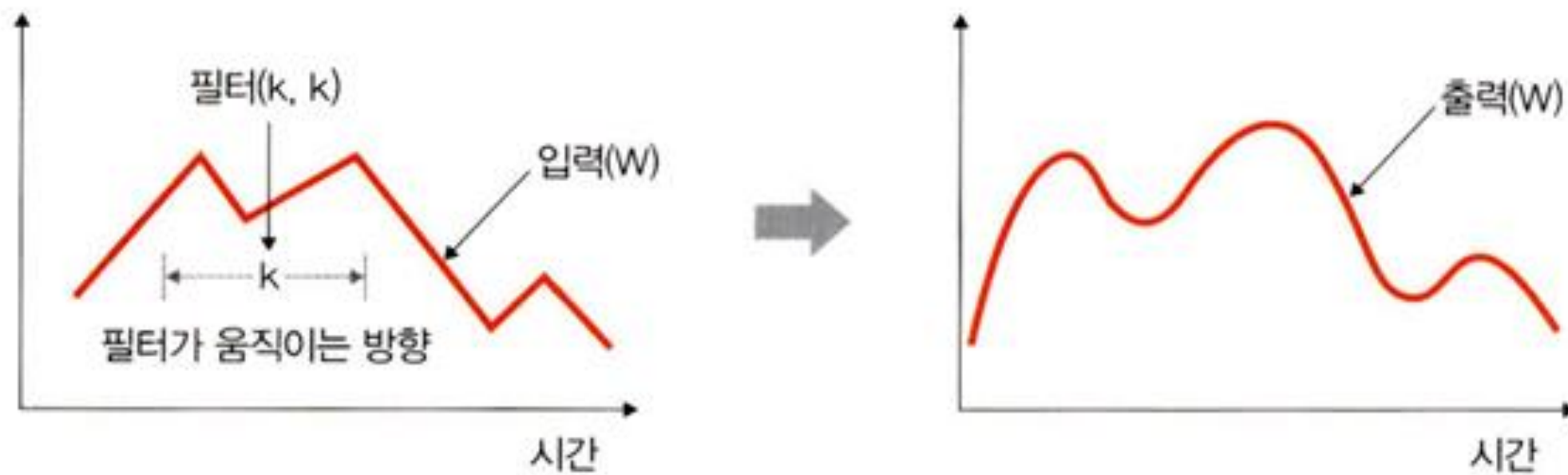
출력층 output layer

- Softmax 함수를 사용하여 입력받은 값을 0 ~ 1 사이의 값으로 출력
- 이미지가 각 레이블에 속할 확률 값이 출력
- 가장 높은 확률 값을 갖는 레이블이 최종 값으로 선정

1D, 2D, 3D 합성곱

1D 합성곱

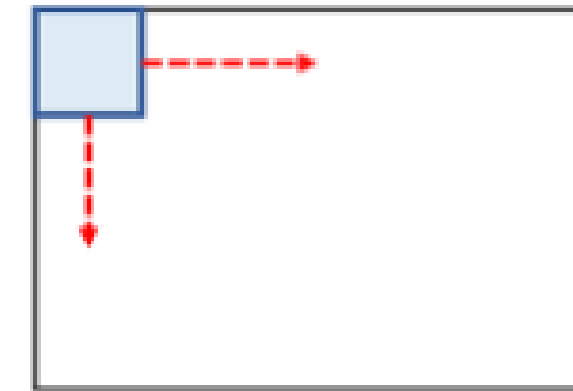
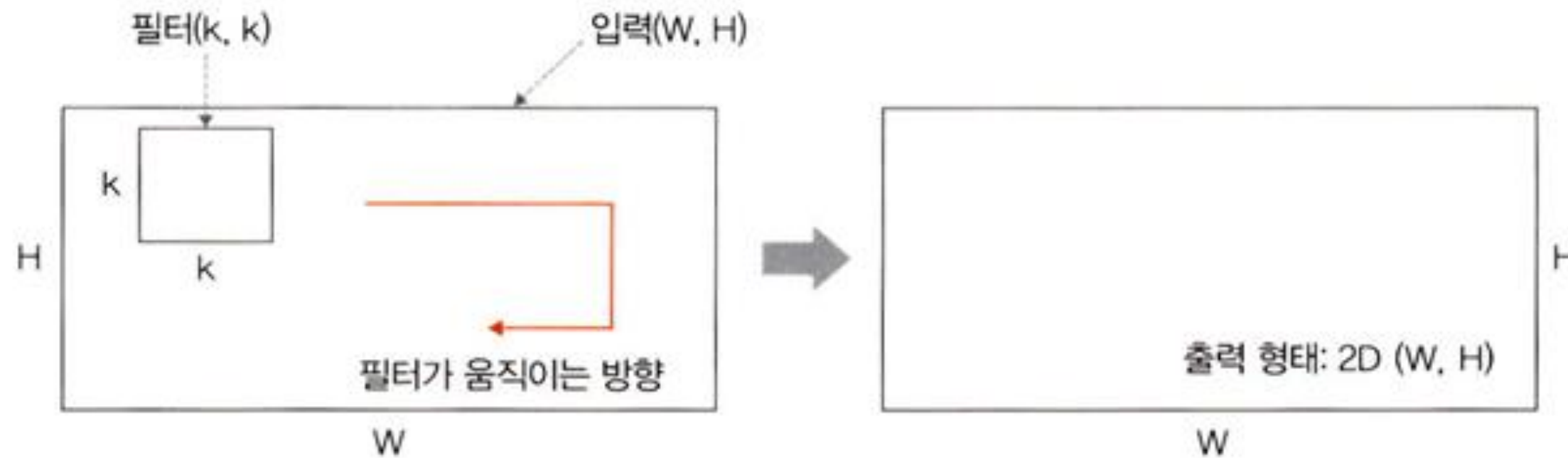
- 시간을 축으로, 좌우로만 이동
- 입력(W) + 필터(k) → 출력(W), 1D 배열
- 그래프 곡선을 완화할 때 많이 사용됨



1D, 2D, 3D 합성곱

2D 합성곱

- 방향 2개로 움직임
- 입력(W, H) + 필터(k, k) → 출력(W, H), 2D 행렬

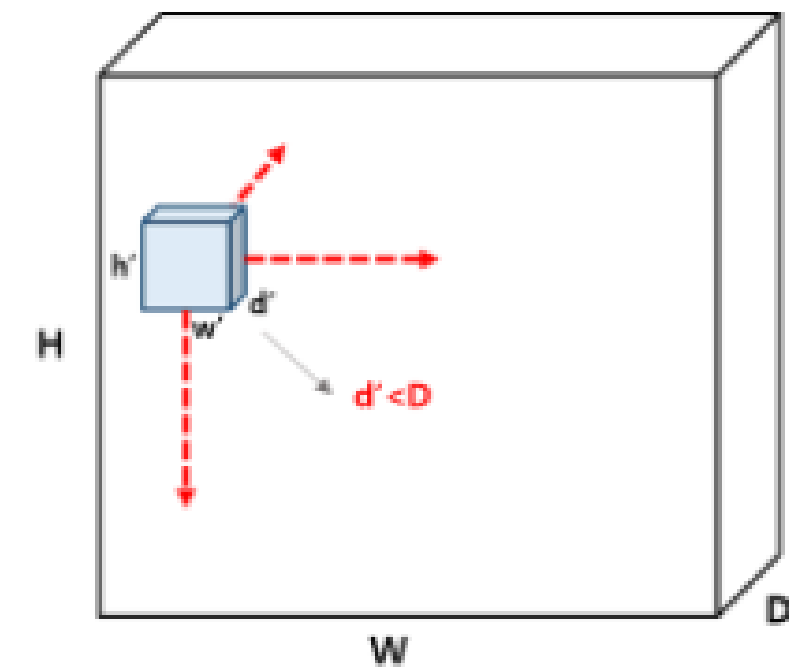
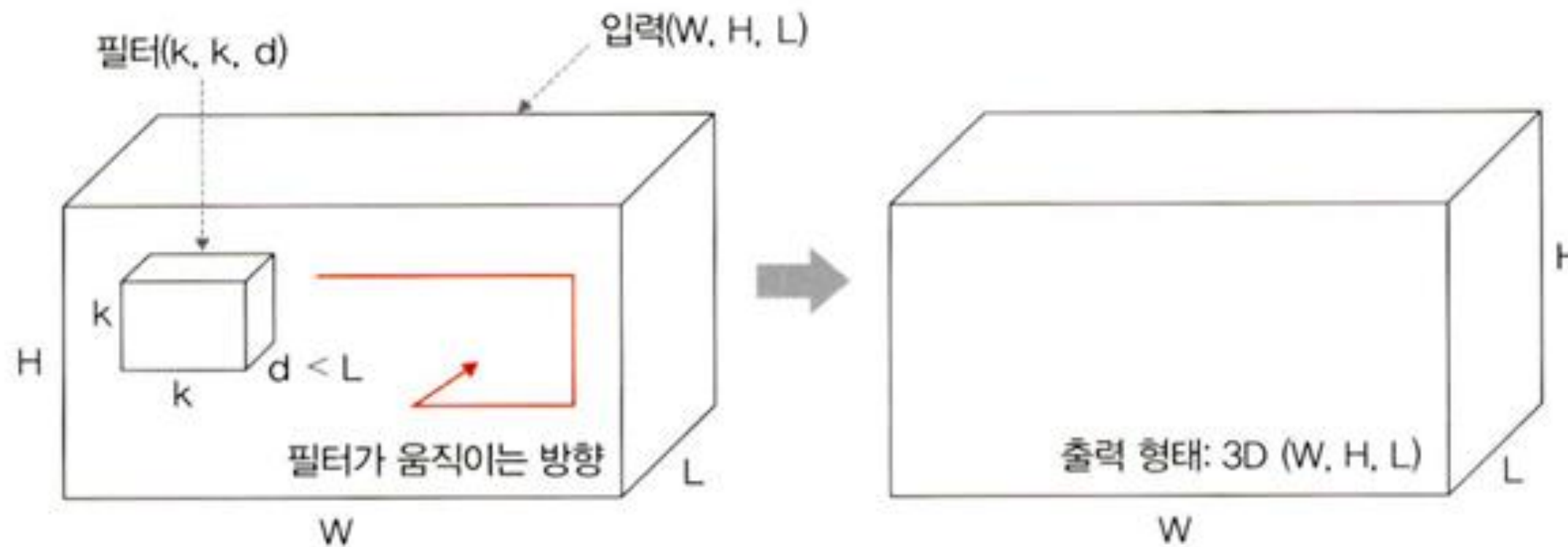


[2D Convolution]

1D, 2D, 3D 합성곱

3D 합성곱

- 방향 3개로 움직임
- 입력(W, H, L) + 필터(k, k, d) $\rightarrow$ 출력(W, H, L), 3D
- $d < L$

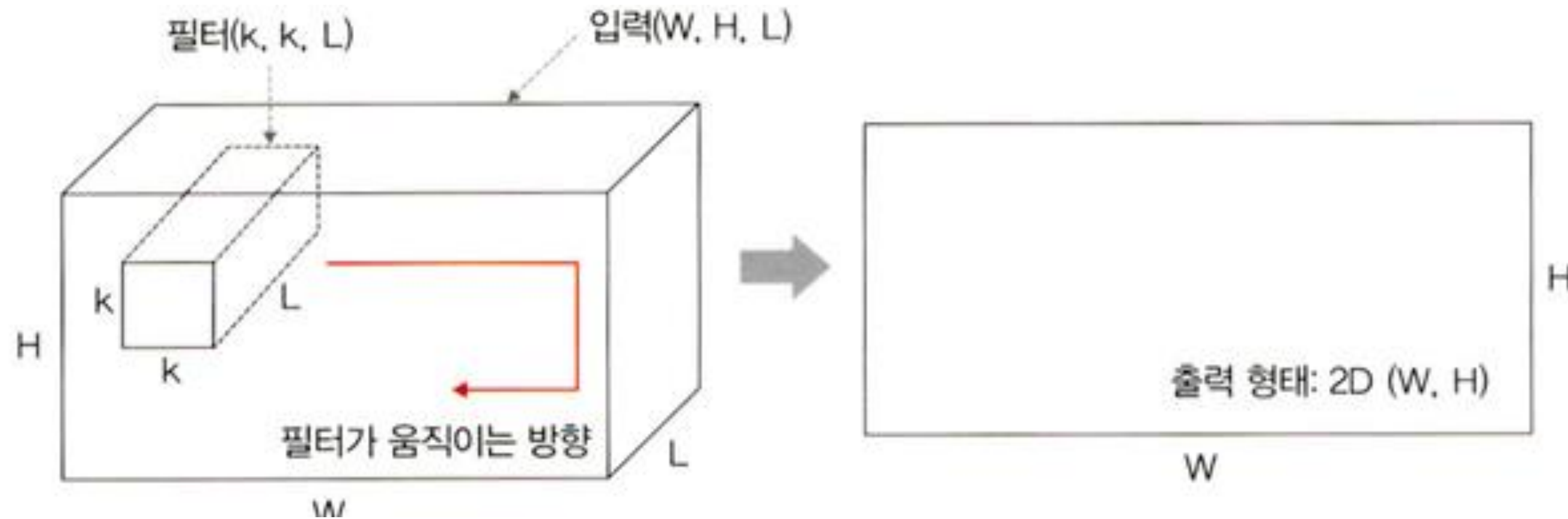


[3D Convolution]

1D, 2D, 3D 합성곱

3D 입력을 갖는 2D 합성곱

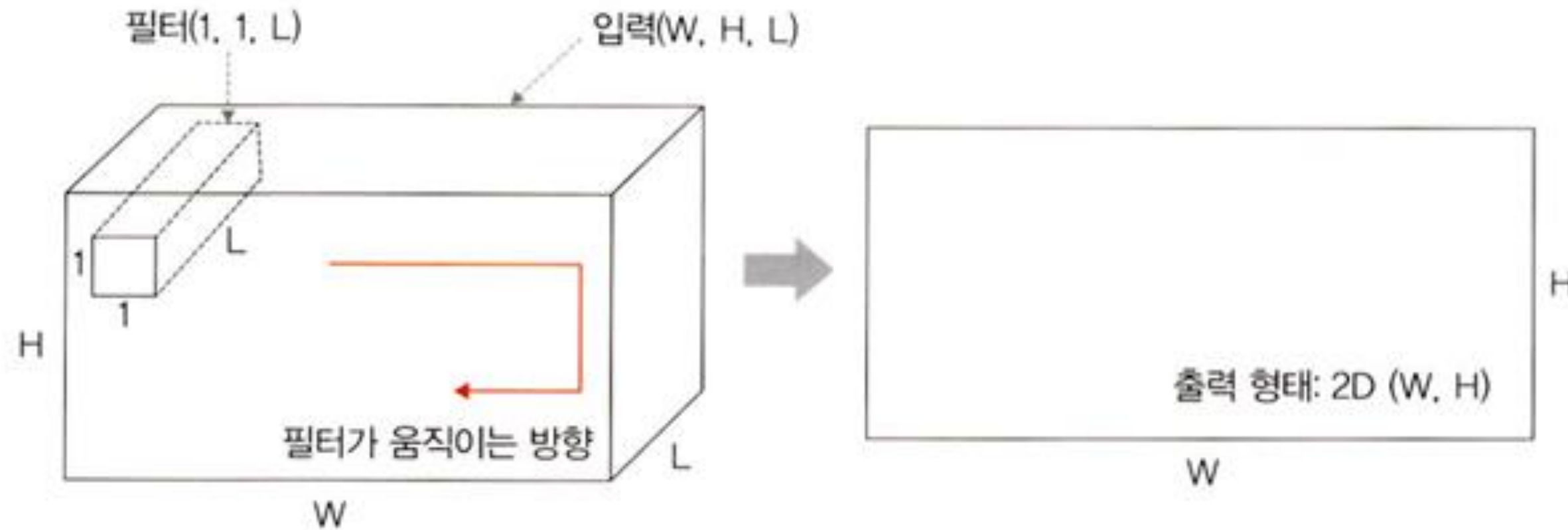
- 입력(W, H, L) + 필터(k, k, L) $\rightarrow$ 출력(W, H), 2D
- 입력 채널(L) = 필터 길이(L)
- 대표적 예시: LeNet-5, VGG



1D, 2D, 3D 합성곱

1 X 1 합성곱

- 입력(W, H, L) + 필터(1, 1, L) → 출력(W, H), 2D
- 채널수를 조정하여 연산량을 감소시킴
- 대표적 예시: GoogLeNet



5.2 합성곱 신경망 맛보기



합성곱 신경망 맛보기

fashion_mnist 데이터셋 사용해 합성곱 신경망 직접 구현하기

0. fashion_mnist 데이터셋?

Torchvision에 내장된 예제 데이터로 운동화, 셔츠, 샌들 같은 작은 이미지의 모음

10 가지로 분류될 수 있는 28x28 픽셀의 이미지 7만개로 구성

- 훈련 데이터는 0~255 사이의 값을 갖는 28x28 크기의 Numpy 배열
- 레이블(정답) 데이터는 0~9까지의 정수 값을 갖는 배열



```
0 : T-Shirt
1 : Trouser
2 : Pullover
3 : Dress
4 : Coat
5 : Sandal
6 : Shirt
7 : Sneaker
8 : Bag
9 : Ankle Boot
```

합성곱 신경망 맛보기

fashion_mnist 데이터셋 사용해 합성곱 신경망 직접 구현하기

1. 필요한 라이브러리 호출

```
import numpy as np
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
from torch.autograd import Variable
import torch.nn.functional as F

import torchvision
import torchvision.transforms as transforms ----- 데이터 전처리를 위해 사용하는 라이브러리
from torch.utils.data import Dataset, DataLoader
```

2. CPU/GPU 장치 확인

```
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
```

- GPU를 사용할 수 있는 환경일 경우 GPU 사용 (권장),
- 사용할 수 없는 환경일 경우 CPU 사용

합성곱 신경망 맛보기

fashion_mnist 데이터셋 사용해 합성곱 신경망 직접 구현하기

3. 데이터셋 내려받기

```
train_dataset = torchvision.datasets.FashionMNIST("../chap05/data", download=True,  
                                                  transform=transforms.Compose([transforms.ToTensor()]))) ----- ①  
test_dataset = torchvision.datasets.FashionMNIST("../chap05/data", download=True,  
                                                  train=False, transform=transforms.Compose([transforms.ToTensor()])))  
                                                    앞에서 훈련 데이터셋을 내려받았다면 여기에서는 테스트 데이터셋을 내려받습니다.
```

Parameters

→ torchvision.datasets.FashionMNIST(① "../chap05/data", ② download=True,
 ③ transform=transforms.Compose([transforms.ToTensor()])))

- fashion_mnist은 토치비전에서 다운로드 가능

- (a): FashionMNIST를 내려받을 위치
- (b): download=True로 변경하면, (a)의 위치에 해당 데이터셋이 있는지 확인한 후 내려받음
- (c): transform: 이미지를 텐서(0 ~ 1)로 변경

합성곱 신경망 맛보기

fashion_mnist 데이터셋 사용해 합성곱 신경망 직접 구현하기

3. 내려받은 데이터셋(fashion_mnist)을 DataLoader에 전달

```
train_loader = torch.utils.data.DataLoader(train_dataset,  
                                           batch_size=100) ----- ① → torch.utils.data.DataLoader(train_dataset, batch_size=100)  
test_loader = torch.utils.data.DataLoader(test_dataset,  
                                           batch_size=100)
```

Parameters

(a) (b)

(a): 데이터를 불러올 데이터셋
(b): 데이터의 배치 사이즈 지정

torch.utils.data.DataLoader()를 사용하여

- 원하는 크기의 배치 단위로 데이터 불러오거나,
- 순서가 무작위로 섞이도록 할 수 있음

합성곱 신경망 맛보기

fashion_mnist 데이터셋 사용해 합성곱 신경망 직접 구현하기

4. 분류에 사용될 클래스 정의

```
labels_map = {0 : 'T-Shirt', 1 : 'Trouser', 2 : 'Pullover', 3 : 'Dress', 4 : 'Coat',  
5 : 'Sandal', 6 : 'Shirt', 7 : 'Sneaker', 8 : 'Bag', 9 : 'Ankle Boot'} ----- 열 개의 클래스
```

```
fig = plt.figure(figsize=(8,8)); ----- 출력할 이미지의 가로세로 길이로 단위는 inch
```

```
columns = 4;
```

```
rows = 5;
```

```
for i in range(1, columns*rows +1):
```

```
    img_xy = np.random.randint(len(train_dataset)); ----- ①
```

```
    img = train_dataset[img_xy][0][0,:,:] ----- ②
```

```
    fig.add_subplot(rows, columns, i)
```

```
    plt.title(labels_map[train_dataset[img_xy][1]])
```

```
    plt.axis('off')
```

```
    plt.imshow(img, cmap='gray')
```

```
plt.show() ----- 20개의 이미지 데이터를 시각적으로 표현
```

(1) np.random()으로 무작위 데이터 생성

random.randint(len(train_dataset)):
0~(train_dataset의 길이)의 분포에서 1개의 랜덤한 숫자 생성

=> 랜덤한 1개의 데이터셋 이미지 선택

import numpy as np

np.random.randint(10)

→ 0~10의 임의의 숫자 출력

np.random.randint(1, 10)

→ 1~9의 임의의 숫자 출력

np.random.rand(8)

np.random.rand(4, 2)

→ 0~1 사이의 정규표준분포 난수를 행렬로 (1x8) 출력

→ 0~1 사이의 정규표준분포 난수를 행렬로 (4x2) 출력

np.random.randn(8)

np.random.randn(4, 2)

→ 평균 0, 표준편차 1인 가우시안 정규분포 난수를 행렬로 (1x8) 출력

→ 평균 0, 표준편차 1인 가우시안 정규분포 난수를 행렬로 (4x2) 출력

합성곱 신경망 맛보기

fashion_mnist 데이터셋 사용해 합성곱 신경망 직접 구현하기

4. 분류에 사용될 클래스 정의

```
labels_map = {0 : 'T-Shirt', 1 : 'Trouser', 2 : 'Pullover', 3 : 'Dress', 4 : 'Coat',  
5 : 'Sandal', 6 : 'Shirt', 7 : 'Sneaker', 8 : 'Bag', 9 : 'Ankle Boot'} ----- 열 개의 클래스
```

```
fig = plt.figure(figsize=(8,8)); ----- 출력할 이미지의 가로세로 길이로 단위는 inch
```

```
columns = 4;
```

```
rows = 5;
```

```
for i in range(1, columns*rows +1):
```

```
    img_xy = np.random.randint(len(train_dataset)); ----- ①
```

```
    img = train_dataset[img_xy][0][0,:,:] ----- ②
```

```
    fig.add_subplot(rows, columns, i)
```

```
    plt.title(labels_map[train_dataset[img_xy][1]])
```

```
    plt.axis('off')
```

```
    plt.imshow(img, cmap='gray')
```

```
plt.show() ----- 20개의 이미지 데이터를 시각적으로 표현
```

(2) `train_dataset[img_xy][0][0,:,:]`

학습 데이터셋에서 무작위로 선택한 흑백 이미지 한 장의 전체 픽셀을 가져옴

- `train_dataset[img_xy]`: (3차원 이미지 텐서, 정답 레이블(=9))

- `train_dataset[img_xy][0]`: [
 [픽셀, 픽셀, ...],
 [픽셀, 픽셀, ...],
 ...
], -> 28x28 픽셀의 이미지
]



합성곱 신경망 맛보기

fashion_mnist 데이터셋 사용해 합성곱 신경망 직접 구현하기

5. 심층 신경망 모델 생성

```
class FashionDNN(nn.Module):
    def __init__(self): ----- ①
        super(FashionDNN, self).__init__()
        self.fc1 = nn.Linear(in_features=784, out_features=256) ----- ②
        self.drop = nn.Dropout(0.25) ----- ③
        self.fc2 = nn.Linear(in_features=256, out_features=128)
        self.fc3 = nn.Linear(in_features=128, out_features=10)

    def forward(self, input_data): ----- ④
        out = input_data.view(-1, 784) ----- ⑤
        out = F.relu(self.fc1(out)) ----- ⑥
        out = self.drop(out)
        out = F.relu(self.fc2(out))
        out = self.fc3(out)
        return out
```

(1) def __init__(self)

- 클래스 형태의 모델은 항상 torch.nn.Module을 상속받음
- __init__(): 객체가 갖는 속성 값 초기화
- super(FashionDNN, self).__init__(): FashionDNN이라는 부모 클래스를 상속받음

*객체(object):

- 메모리를 할당받아 프로그램에서 사용되는 모든 데이터 (변수/함수 등)
- 사용 방식: 객체명 = 클래스명()

*클래스(class):

- 함수 및 관련된 변수까지도 한번에 묶어서 관리하고 재사용 가능한 일종의 틀

합성곱 신경망 맛보기

fashion_mnist 데이터셋 사용해 합성곱 신경망 직접 구현하기

5. 심층 신경망 모델 생성

```
class FashionDNN(nn.Module):
    def __init__(self): ----- ①
        super(FashionDNN, self).__init__()
        self.fc1 = nn.Linear(in_features=784, out_features=256) ----- ②
        self.drop = nn.Dropout(0.25) ----- ③
        self.fc2 = nn.Linear(in_features=256, out_features=128)
        self.fc3 = nn.Linear(in_features=128, out_features=10)

    def forward(self, input_data): ----- ④
        out = input_data.view(-1, 784) ----- ⑤
        out = F.relu(self.fc1(out)) ----- ⑥
        out = self.drop(out)
        out = F.relu(self.fc2(out))
        out = self.fc3(out)
        return out
```



(2) `nn.Linear(in_features=784, out_features=256)`

- nn: 딥러닝 모델(네트워크) 구성에 필요한 모듈이 모여있는 패키지
- Linear: 단순 선형 회귀 모델을 만들 때 사용
- in_features: 입력의 크기(input size)
- out_features: 출력의 크기 (output size)
=> forward() 연산의 크기가 됨

합성곱 신경망 맛보기

fashion_mnist 데이터셋 사용해 합성곱 신경망 직접 구현하기

5. 심층 신경망 모델 생성

```
class FashionDNN(nn.Module):
    def __init__(self): ----- ①
        super(FashionDNN, self).__init__()
        self.fc1 = nn.Linear(in_features=784, out_features=256) ----- ②
        self.drop = nn.Dropout(0.25) ----- ③
        self.fc2 = nn.Linear(in_features=256, out_features=128)
        self.fc3 = nn.Linear(in_features=128, out_features=10)

    def forward(self, input_data): ----- ④
        out = input_data.view(-1, 784) ----- ⑤
        out = F.relu(self.fc1(out)) ----- ⑥
        out = self.drop(out)
        out = F.relu(self.fc2(out))
        out = self.fc3(out)
        return out
```

(3) torch.nn.Dropout(p)

- p만큼의 비율로 텐서의 값이 0이 됨
- 0이 되지 않는 값들은 기존 값에 $(1/(1-p))$ 만큼 곱해져 커짐

e.g.) $p=0.3 \Rightarrow$ 전체 값 중 0.3의 확률로 0이 되며,
0.7에 해당하는 값은 $(1/(1-0.7))=3.33$ 만큼 커짐

해당 예제에서는 $p=0.25$ 로, 전체 값 중 0.25 확률로 0이 되며
0.75에 해당하는 값은 $(1/(1-0.75))=4$ 만큼 커짐

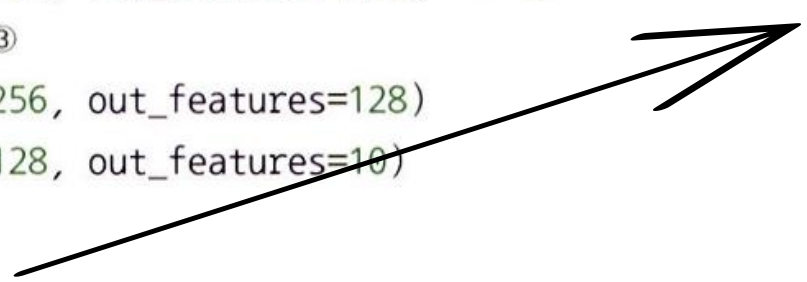
합성곱 신경망 맛보기

fashion_mnist 데이터셋 사용해 합성곱 신경망 직접 구현하기

5. 심층 신경망 모델 생성

```
class FashionDNN(nn.Module):
    def __init__(self): ----- ①
        super(FashionDNN, self).__init__()
        self.fc1 = nn.Linear(in_features=784, out_features=256) ----- ②
        self.drop = nn.Dropout(0.25) ----- ③
        self.fc2 = nn.Linear(in_features=256, out_features=128)
        self.fc3 = nn.Linear(in_features=128, out_features=10)

    def forward(self, input_data): ----- ④
        out = input_data.view(-1, 784) ----- ⑤
        out = F.relu(self.fc1(out)) ----- ⑥
        out = self.drop(out)
        out = F.relu(self.fc2(out))
        out = self.fc3(out)
        return out
```



(4) def forward(self, input_data)

- 모델이 학습 데이터를 입력받아 순전파 학습을 진행, 객체를 데이터와 함께 호출하면 자동으로 실행됨.
(반드시 forward라는 이름의 함수여야 함)
- 순전파 연산: $H(x)$ 식에 입력 x 로부터 예측된 y 값을 얻는 것

합성곱 신경망 맛보기

fashion_mnist 데이터셋 사용해 합성곱 신경망 직접 구현하기

5. 심층 신경망 모델 생성

```
class FashionDNN(nn.Module):
    def __init__(self): ----- ①
        super(FashionDNN, self).__init__()
        self.fc1 = nn.Linear(in_features=784, out_features=256) ----- ②
        self.drop = nn.Dropout(0.25) ----- ③
        self.fc2 = nn.Linear(in_features=256, out_features=128)
        self.fc3 = nn.Linear(in_features=128, out_features=10)

    def forward(self, input_data): ----- ④
        out = input_data.view(-1, 784) ----- ⑤
        out = F.relu(self.fc1(out)) ----- ⑥
        out = self.drop(out)
        out = F.relu(self.fc2(out))
        out = self.fc3(out)
        return out
```

(5) `input_data.view()`

- `view`: 텐서의 크기(shape)를 변경해 주는 역할
(= 넘파이 reshape)

`input_data.view(-1, 784)`: `input_data`를 (?, 784) 크기의
2차원 텐서로 변경

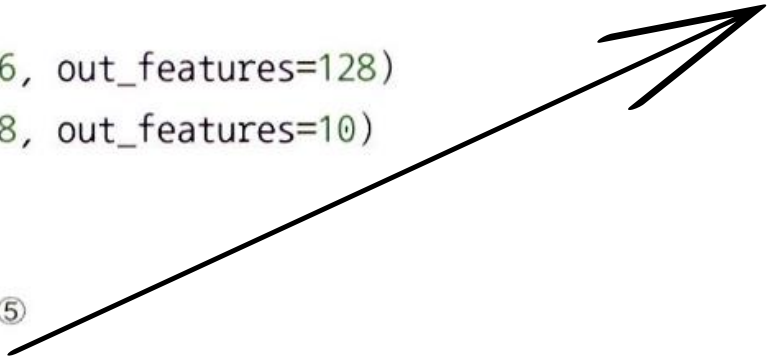
합성곱 신경망 맛보기

fashion_mnist 데이터셋 사용해 합성곱 신경망 직접 구현하기

5. 심층 신경망 모델 생성

```
class FashionDNN(nn.Module):
    def __init__(self): ----- ①
        super(FashionDNN, self).__init__()
        self.fc1 = nn.Linear(in_features=784, out_features=256) ----- ②
        self.drop = nn.Dropout(0.25) ----- ③
        self.fc2 = nn.Linear(in_features=256, out_features=128)
        self.fc3 = nn.Linear(in_features=128, out_features=10)

    def forward(self, input_data): ----- ④
        out = input_data.view(-1, 784) ----- ⑤
        out = F.relu(self.fc1(out)) ----- ⑥
        out = self.drop(out)
        out = F.relu(self.fc2(out))
        out = self.fc3(out)
        return out
```



(6) F.relu(): ReLU 활성화 함수 적용

활성화 함수를 사용하는 위치에 따른 함수 지정 방법

- F.relu(): forward() 함수에서 정의
- nn.ReLU(): __init__() 함수에서 정의

합성곱 신경망 맛보기

fashion_mnist 데이터셋 사용해 합성곱 신경망 직접 구현하기

두 활성화함수의 사용 방법 차이 - 자세히

1. nn을 사용하는 코드

```
import torch
import torch.nn as nn

inputs = torch.randn(64, 3, 244, 244)
conv = nn.Conv2d(in_channels=3, out_channels=64, kernel_size=3, padding=1)
outputs = conv(inputs)
layer = nn.Conv2d(1, 1, 3)
```

세 개의 채널이 입력되어 64개의 채널이 출력되기
위한 연산으로 3×3 크기의 커널을 사용

2. nn.functional(=F.xx)을 사용하는 코드

```
import torch.nn.functional as F

inputs = torch.randn(64, 3, 244, 244)
weight = torch.randn(64, 3, 3, 3)
bias = torch.randn(64)
outputs = F.conv2d(inputs, weight, bias, padding=1)
```

-> nn.Conv2d에서 input_channel과 output_channel을 사용해서 연산했다면 functional.conv2d는 입력(input)과 가중치(weight)를 직접 넣어주어야 함

*nn.xx와 nn.functional.xx의 사용 방법 비교 표

구분	nn.xx	nn.functional.xx
형태	nn.Conv2d: 클래스 nn.Module 클래스를 상속받아 사용	nn.functional.conv2d: 함수 def function (input)으로 정의된 순수한 함수
호출 방법	먼저 하이퍼파라미터를 전달한 후 함수 호출을 통해 데이터 전달	함수를 호출할 때 하이퍼파라미터, 데이터 전달
위치	nn.Sequential 내에 위치	nn.Sequential에 위치할 수 없음
파라미터	파라미터를 새로 정의할 필요 없음	가중치를 수동으로 전달해야 할 때마다 자체 가중치 를 정의

심층 신경망에서 필요한 파라미터 정의

코드 5-7 심층 신경망에서 필요한 파라미터 정의

```
learning_rate = 0.001;
model = FashionDNN();
model.to(device)

criterion = nn.CrossEntropyLoss(); ----- 분류 문제에서 사용하는 손실 함수
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate); ----- ①
print(model)
```

옵티마이저를 위한 경사 하강법은 **Adam**, 학습률은 **0.001**을 사용

```
FashionDNN(
  (fc1): Linear(in_features=784, out_features=256, bias=True)
  (drop): Dropout(p=0.25, inplace=False)
  (fc2): Linear(in_features=256, out_features=128, bias=True)
  (fc3): Linear(in_features=128, out_features=10, bias=True)
)
```

생성한 심층 신경망 모델을 보여 줌

심층 신경망을 이용한 모델 학습

코드 5-8 심층 신경망을 이용한 모델 학습

```
num_epochs = 5
count = 0

loss_list = [] ----- ①
iteration_list = []
accuracy_list = []

predictions_list = []
labels_list = []
```

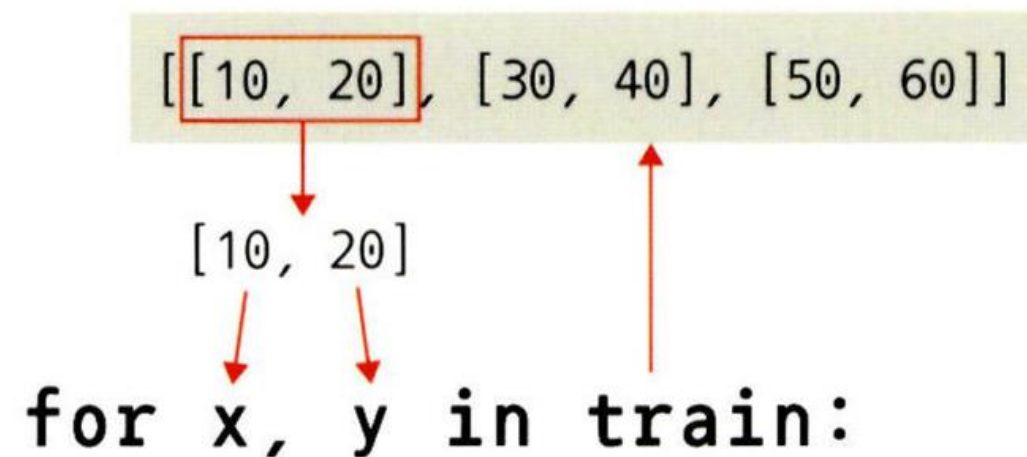
배치 사이즈는 100, 데이터는 60000개
-> 1 에포크에 600 iteration (count)
-> 5 에포크 도니까 3000 iteration (count)

①과 같이 비어 있는 배열이나 행렬을 만들고
이후에 **append** 메서드를 이용하여 데이터를
하나씩 추가

심층 신경망을 이용한 모델 학습

```
for epoch in range(num_epochs):  반복문이 한 바퀴 돌 때 전체 학습 데이터를 한 번 학습
    for images, labels in train_loader: ----- ②  반복문이 한 바퀴 돌 때 배치 사이즈인 사진 100장 학습
        images, labels = images.to(device), labels.to(device) ----- ③
        images, labels = images.to(device), labels.to(device) ----- ③
```

▼ 그림 5-26 for in 구문



for 구문을 사용하여 **레코드**(행, 가로줄)를 하나씩 가져옴

이때 for x, y in train:과 같이 in 앞에 변수를 두 개 지정해 주면 **레코드**에서 **요소 두 개**를 꺼내 오겠다는 의미임

심층 신경망을 이용한 모델 학습

모델에 넣을 데이터 준비

```
train = Variable(images.view(100, 1, 28, 28)) ----- ④  
labels = Variable(labels)
```

forward 실행 -> 손실값 계산 -> 역전파 통해 손실에 대한
가중치와 편향의 기울기 계산 -> 가중치와 편향 수정

```
outputs = model(train) ----- 학습 데이터를 모델에 적용  
loss = criterion(outputs, labels)  
optimizer.zero_grad()  
loss.backward()  
optimizer.step()  
count += 1
```

Autograd는 자동 미분을 수행하며, 자동 미분에 대한 값을 저장하기 위해 **테이프(tape)**를 사용함

순전파(forward) 단계에서 테이프는 수행하는 모든 연산을 저장하고 **역전파(backward)** 단계에서 저장된 값들을 꺼내서 사용함

Autograd는 **Variable**을 사용해서 역전파를 위한 **미분 값**을 자동으로 계산해줌

심층 신경망을 이용한 모델 학습

50번 학습할 때마다 테스트 정확도 계산, 기록

`if not (count % 50):` ----- count를 50으로 나누었을 때 나머지가 0이 아니라면 실행

`total = 0`

`correct = 0`

`for images, labels in test_loader:`

`images, labels = images.to(device), labels.to(device)`

`labels_list.append(labels)`

`test = Variable(images.view(100, 1, 28, 28))`

`outputs = model(test)`

`predictions = torch.max(outputs, 1)[1].to(device)`

`outputs.shape` 은 [100, 10] 이므로 dim = 1 의 max를 찾으라는 의미
코드의 [1]은 최고 점수 대신 번호를 선택

`predictions_list.append(predictions)`

`correct += (predictions == labels).sum()` 맞힌 수와 전체 사진 수를 누적

`total += len(labels)`

`accuracy = correct * 100 / total` ----- ⑤

`loss_list.append(loss.data)` ----- ①'

`iteration_list.append(count)`

`accuracy_list.append(accuracy)`

500번 학습할 때마다 화면에 결과 출력

`if not (count % 500):`

`print("Iteration: {}, Loss: {}, Accuracy: {}".format(count, loss.data, accuracy))`

심층 신경망을 이용한 모델 학습

- 분류 문제에 대한 정확도는 전체 예측에 대한 **정확한 예측의 비율**로 표현할 수 있음

$\text{classification accuracy} = \text{correct predictions} / \text{total predictions}$

- 결과에 100을 곱하여 **백분율**로 표시

$\text{classification accuracy} = \text{correct predictions} / \text{total predictions} * 100$

- 분류 문제에 대한 정확도는 다음과 같이 값을 반전시켜 **오분류율** 또는 **오류율**로 표현할 수 있음

$\text{error rate} = (1 - (\text{correct predictions} / \text{total predictions})) * 100$

분류 문제에서 클래스가 세 개 이상일 때 정확도 값이 모든 클래스가 동등하게 고려된 것인지, 특정 클래스의 분류가 높았던 것인지에 대해 알 수 없음에 유의해야 함.

또한, 모든 데이터를 특정 클래스에 속한다고 예측해도 **높은 정확도**가 나오기 때문에 데이터 특성에 따라 정확도를 잘 관측해야 함.

합성곱 네트워크 생성

코드 5-9 합성곱 네트워크 생성

```
class FashionCNN(nn.Module):
    def __init__(self):
        super(FashionCNN, self).__init__()
        self.layer1 = nn.Sequential( ----- ①
            nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1), ----- ②
            nn.BatchNorm2d(32), ----- ③
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2) ----- ④
        )
        self.layer2 = nn.Sequential(
            nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )
        self.fc1 = nn.Linear(in_features=64*6*6, out_features=600) ----- ⑤
        self.drop = nn.Dropout2d(0.25)
        self.fc2 = nn.Linear(in_features=600, out_features=120)
```

```
self.fc3 = nn.Linear(in_features=120, out_features=10) -----
                                                                마지막 계층의 out_features는 클래스 개수를 의미
```

```
def forward(self, x):
    out = self.layer1(x)
    out = self.layer2(out)
    out = out.view(out.size(0), -1) ----- ⑥
    out = self.fc1(out)
    out = self.drop(out)
    out = self.fc2(out)
    out = self.fc3(out)
    return out
```

합성곱 네트워크 생성

nn.Sequential

```
self.layer1 = nn.Sequential( ----- ①
    nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1), ----- ②
    nn.BatchNorm2d(32), ----- ③
    nn.ReLU(),
    nn.MaxPool2d(kernel_size=2, stride=2) ----- ④
)
```

nn.Sequential을 사용하면 `__init__()`에서 사용할 네트워크 모델들을 정의해 줄 뿐만 아니라 `forward()` 함수에서 구현될 순전파를 **계층(layer)** 형태로 좀 더 가독성이 뛰어난 코드로 작성할 수 있음.

nn.Sequential은 계층을 차례로 쌓을 수 있도록 $Wx + b$ 와 같은 수식과 활성화 함수를 연결해 주는 역할

특히 데이터가 각 계층을 순차적으로 지나갈 때 사용하면 좋은 방법

합성곱 네트워크 생성

합성곱층(conv layer)

합성곱층(conv layer)은 합성곱 연산을 통해서 **이미지의 특징**을 추출함.

합성곱이란 커널(또는 필터)이라는 **$n \times m$** 크기의 행렬이 높이(height)×너비(width) 크기의 이미지를 처음부터 끝까지 훑으면서 각 원소 값끼리 곱한 후 모두 더한 값을 출력함.

커널은 일반적으로 **3×3** 이나 **5×5** 를 사용하며 파라미터는 다음과 같음.

```
nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1)
```

㉠

㉡

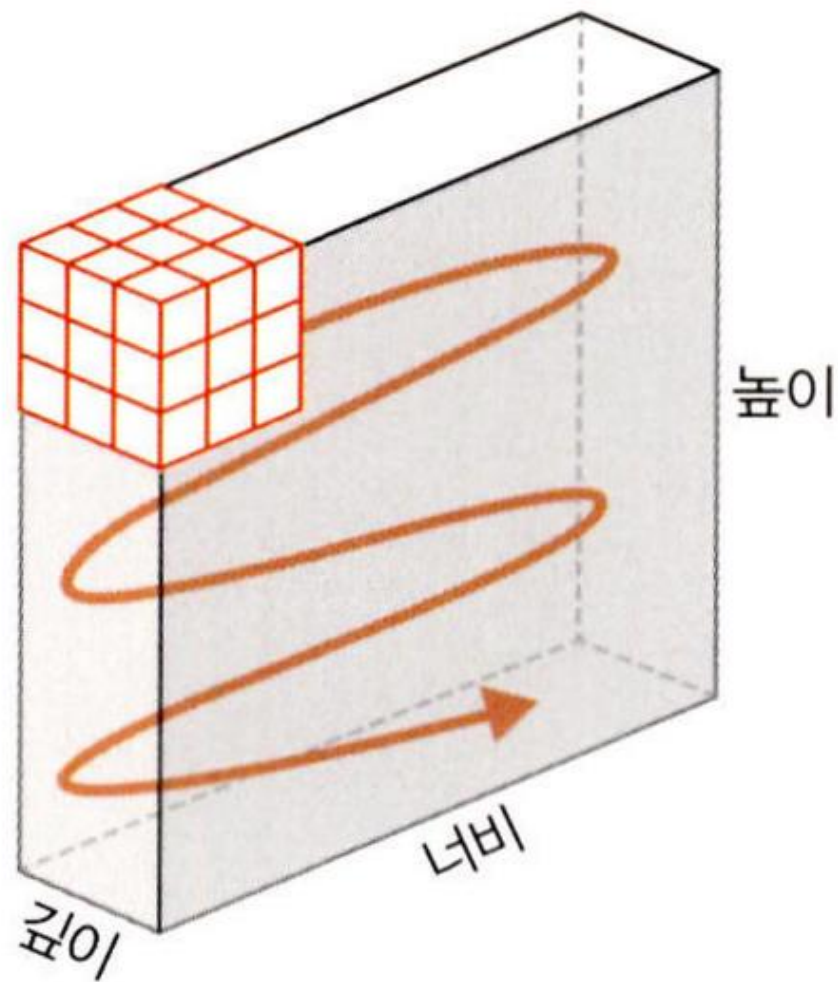
㉢

㉣

합성곱 네트워크 생성

채널

▼ 그림 5-27 채널



in_channels: 입력 채널의 수를 의미함.

흑백 이미지는 1, RGB 값을 가진 이미지는 3을 가진 경우가 많음.

out_channels: 출력 채널의 수를 의미함.

흑백 이미지일 경우 이미지 데이터는 **w×h** 형태의 행렬로 표현됨

컬러 이미지는 일반적으로 **Red, Green, Blue**라는 세 개의 채널을 가짐

이때 컬러 이미지는 **w×h×c** 형태의 행렬로 표현될 수 있음
(c는 채널 수를 의미함).

3차원으로 생각하면 채널은 결국 **깊이(depth)**를 의미

합성곱 네트워크 생성

kernel_size · padding

```
nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1)
```

(a) (b) (c) (d)

kernel_size: 커널 크기를 의미하며 필터라고도 함.

커널은 이미지 특징을 찾아내기 위한 공용 파라미터이며, CNN에서 **학습 대상**은 필터 파라미터가 됨.

커널은 입력 데이터를 **스트라이드** 간격으로 순회하면서 합성곱을 계산함.

padding: 패딩 크기를 의미하는 것으로 **출력 크기**를 조정하기 위해 입력 데이터 주위에 **0**을 채움.

패딩 값이 클수록 **출력 크기**도 커짐.

합성곱 네트워크 생성

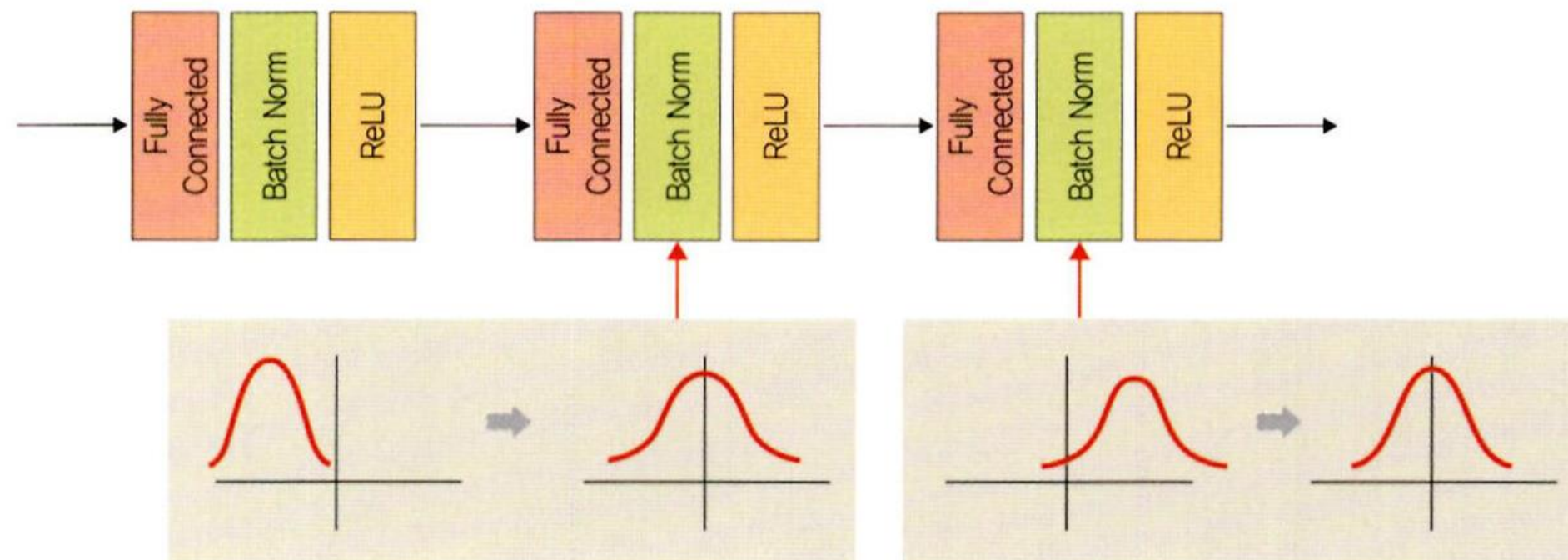
BatchNorm2d

BatchNorm2d는 학습 과정에서 각 배치 단위별로 데이터가 다양한 분포를 가지더라도 **평균과 분산**을 이용하여 정규화하는 것을 의미

다음 그림을 보면 배치 단위나 계층에 따라 입력 값의 분포가 모두 다르지만 정규화를 통해 분포를 가우시안 형태로 만듦.

그러면 **평균은 0, 표준편차는 1**로 데이터의 분포가 조정됨.

▼ 그림 5-28 BatchNorm2d



합성곱 네트워크 생성

MaxPool2d

MaxPool2d는 **이미지 크기를 축소**시키는 용도로 사용

풀링 계층은 합성곱층의 출력 데이터를 입력으로 받아서 출력 데이터(activation map)의 크기를 줄이거나 특정 데이터를 강조하는 용도로 사용됨.

풀링 계층을 처리하는 방법으로는 **최대 풀링**(max pooling)과 **평균 풀링**(average pooling), **최소 풀링**(min pooling)이 있음

```
nn.MaxPool2d(kernel_size=2, stride=2)
```

①

②

kernel_size: $m \times n$ 행렬로 구성된 가중치

stride: 입력 데이터에 커널을 적용할 때 이동할 간격을 의미하는데, **스트라이드 값**이 커지면 출력 크기는 작아짐.

합성곱 네트워크 생성

완전연결층(fully connected layer)

클래스를 분류하기 위해서는 이미지 형태의 데이터를 **배열 형태**로 변환하여 작업해야 함.

이때 Conv2d에서 사용하는 하이퍼파라미터 값들에 따라 출력 크기(output size)가 달라짐.

즉, **패딩과 스트라이드**의 값에 따라 출력 크기가 달라짐.

이렇게 줄어든 출력 크기는 최종적으로 분류를 담당하는 **완전연결층**으로 전달됨.

```
nn.Linear(in_features=64*6*6, out_features=600)
```

Ⓐ

Ⓑ

합성곱 네트워크 생성

in_features · out_features

```
nn.Linear(in_features=64*6*6, out_features=600)
```

(a) (b)

in_features: 입력 데이터의 크기를 의미

중요한 것은 이전까지 수행했던 Conv2d, MaxPool2d는 이미지 데이터를 입력으로 받아 처리했음.

하지만 그 출력 결과를 완전연결층으로 보내기 위해서는 **1차원**으로 변경해 주어야 함

out_features: 출력 데이터의 크기를 의미

합성곱 네트워크 생성

Conv2d 계층에서의 출력 크기 구하는 공식

$$\text{출력 크기} = (W - F + 2P) / S + 1$$

W: 입력 데이터의 크기(input_volume_size)

F: 커널 크기(kernel_size)

P: 패딩 크기(padding_size)

S: 스트라이드(strides)

```
nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1)
```

합성곱 네트워크 생성

MaxPool2d 계층에서의 출력 크기 구하는 공식

$$\text{출력 크기} = IF/F$$

IF: 입력 필터의 크기

F: 커널 크기(kernel_size)

```
nn.MaxPool2d(kernel_size=2, stride=2)
```

Ⓐ

Ⓑ

합성곱 네트워크를 위한 파라미터 정의

코드 5-10 합성곱 네트워크를 위한 파라미터 정의

```
learning_rate = 0.001;
model = FashionCNN();

model.to(device)

criterion = nn.CrossEntropyLoss();
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate);
print(model)
```

합성곱 네트워크를 위한 파라미터 정의

```
FashionCNN(  
  (layer1): Sequential(  
    (0): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
    (2): ReLU()  
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)  
  )  
  (layer2): Sequential(  
    (0): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1))  
    (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
    (2): ReLU()  
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)  
  )  
  (fc1): Linear(in_features=2304, out_features=600, bias=True)  
  (drop): Dropout2d(p=0.25, inplace=False)  
  (fc2): Linear(in_features=600, out_features=120, bias=True)  
  (fc3): Linear(in_features=120, out_features=10, bias=True)  
)
```

모델 학습 및 성능 평가

모델 학습

모든 준비가 완료되었기 때문에 학습 데이터를 이용하여 모델을 학습시킴.

이 부분은 앞에서 사용했던 코드 5-8과 동일함.

코드 5-11 모델 학습 및 성능 평가

```
num_epochs = 5
count = 0
loss_list = []
iteration_list = []
accuracy_list = []

predictions_list = []
labels_list = []

for epoch in range(num_epochs):
```

```
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        train = Variable(images.view(100, 1, 28, 28))
        labels = Variable(labels)

        outputs = model(train)
        loss = criterion(outputs, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        count += 1
```

모델 학습 및 성능 평가

성능 평가

```
if not (count % 50):
    total = 0
    correct = 0
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        labels_list.append(labels)
        test = Variable(images.view(100, 1, 28, 28))
        outputs = model(test)
        predictions = torch.max(outputs, 1)[1].to(device)
        predictions_list.append(predictions)
        correct += (predictions == labels).sum()
        total += len(labels)
```

```
accuracy = correct * 100 / total
loss_list.append(loss.data)
iteration_list.append(count)
accuracy_list.append(accuracy)
```

```
if not (count % 500):
    print("Iteration: {}, Loss: {}, Accuracy: {}".format(count, loss.data,
        accuracy))
```

```
classification accuracy = correct predictions / total predictions * 100
```

모델 학습 및 성능 평가

모델 훈련 결과

다음과 같은 모델 훈련 결과를 볼 수 있음.

```
Iteration: 500, Loss: 0.43928107619285583, Accuracy: 87.87000274658203%  
Iteration: 1000, Loss: 0.3258974850177765, Accuracy: 88.0999984741211%  
Iteration: 1500, Loss: 0.30115756392478943, Accuracy: 88.38999938964844%  
Iteration: 2000, Loss: 0.1754811853170395, Accuracy: 89.62999725341797%  
Iteration: 2500, Loss: 0.13775354623794556, Accuracy: 89.56999969482422%  
Iteration: 3000, Loss: 0.24831737577915192, Accuracy: 88.94999694824219%
```

모델 학습 및 성능 평가

심층 신경망과 합성곱 신경망

FashionDNN

Iteration: 500, Loss: 0.5627565979957581, Accuracy: 82.70999908447266%
Iteration: 1000, Loss: 0.4662465751171112, Accuracy: 84.4000015258789%
Iteration: 1500, Loss: 0.33023178577423096, Accuracy: 84.41000366210938%
Iteration: 2000, Loss: 0.3131774067878723, Accuracy: 85.05999755859375%
Iteration: 2500, Loss: 0.28404906392097473, Accuracy: 86.26000213623047%
Iteration: 3000, Loss: 0.31252551078796387, Accuracy: 86.4800033569336%

Accuracy: 86.4800033569336%

FashionCNN

Iteration: 500, Loss: 0.43928107619285583, Accuracy: 87.87000274658203%
Iteration: 1000, Loss: 0.3258974850177765, Accuracy: 88.0999984741211%
Iteration: 1500, Loss: 0.30115756392478943, Accuracy: 88.38999938964844%
Iteration: 2000, Loss: 0.1754811853170395, Accuracy: 89.62999725341797%
Iteration: 2500, Loss: 0.13775354623794556, Accuracy: 89.56999969482422%
Iteration: 3000, Loss: 0.24831737577915192, Accuracy: 88.94999694824219%

Accuracy: 88.94999694824219%

심층 신경망과 비교하여 **정확도가 약간** 높음.

실제로 이미지 데이터가 많아지면 단순 심층 신경망으로는 정확한 **특성 추출 및 분류**가 불가능하므로 **합성곱 신경망**을 생성할 수 있도록 학습해야 함.

THANK YOU





2주차 질문

4장: CNN

질문1: 안서영

Q. 우리가 코드 5-9에서 작성한 CNN 코드의 블록 순서는 Conv2d -> BatchNorm2d -> ReLU -> MaxPool2d 입니다. 여기서 BatchNorm을 활성화 함수(ReLU)보다 앞에 두는 이유가 무엇일까요? 만약 순서를 바꿔서 ReLU -> BatchNorm으로 두면 어떤 문제가 생길 수 있을까요?

질문2: 한수연

Q.

Q. 5.1.1에서 DNN은 이미지를 처음부터 1차원으로 flatten하면 공간적 구조를 활용하기 어렵기 때문에, 이를 보완하기 위해 CNN에서는 합성곱층과 풀링층을 통해 특징을 먼저 추출한다고 배웠습니다.
그런데 코드 5-10의 ⑥에서는 CNN도 마지막에 완전연결층으로 넘어가기 위해 feature map을 1차원으로 변환하는 과정이 나오는데, 처음부터 flatten하는 DNN과 합성곱과 풀링을 거친 뒤 flatten하는 CNN은 학습 과정에서 어떤 차이가 생기는지 궁금합니다.
또한 CNN에서 앞서 추출한 공간적 특징이 flatten 이후에도 어떻게 분류에 유효한 정보로 활용되는지 논의해보고 싶습니다.